

EXPERIMENT -2

Student Name: Kapish Gupta

Branch: AIT-CSE (AIML)

Semester: 5

Subject Name: Database Management System

UID: 24BAI70577

Section/Group: 24AIT_KRG-1A

Subject Code: 24CSH-298

- **(A)**

A university administration requires an analytical performance report for each academic unit. Write a PostgreSQL solution to calculate the average marks scored by students grouped by their respective departments. The solution must join the student profile, course subjects, and grade marks tables, returning a breakdown sorted by the highest average marks down to the lowest.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

```
--A.
CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL,
    dept_id INT REFERENCES Departments(dept_id)
);

CREATE TABLE Subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(50) NOT NULL
);

CREATE TABLE Marks (
    student_id INT REFERENCES Students(student_id),
    subject_id INT REFERENCES Subjects(subject_id),
    marks_scored INT NOT NULL,
    PRIMARY KEY (student_id, subject_id)
);
```

EXPERIMENT -2

```

INSERT INTO Departments (dept_id, dept_name) VALUES
(1, 'Computer Science'),
(2, 'Mathematics');

INSERT INTO Students (student_id, student_name, dept_id) VALUES
(101, 'Alex', 1),
(102, 'Bella', 1),
(103, 'Charlie', 2);

INSERT INTO Subjects (subject_id, subject_name) VALUES
(501, 'Data Structures'),
(502, 'Calculus');

INSERT INTO Marks (student_id, subject_id, marks_scored) VALUES
(101, 501, 85),
(101, 502, 90),
(102, 501, 75),
(103, 502, 95);

--SOLUTION
SELECT
    D.DEPT_NAME AS DEPARTMENT,
    AVG(M.MARK_SCORED) AS 'AVERAGE MARKS'
FROM
    DEPARTMENT AS D
JOIN
    STUDENT AS S
ON
    D.DEPT_ID = S.DEPT_ID
JOIN
    MARKS AS M
ON
    M.STUDENT_ID = S.STUDENT_ID
GROUP BY D.DEPT_NAME
ORDER BY 'AVERAGE MARKS' DESC;

```

OUTPUT SCREENSHOTS:

```

48 SELECT
49     D.DEPT_NAME AS DEPARTMENT,
50     ROUND(AVG(M.MARKS_SCORED),2) AS "AVERAGE MARKS"
51 FROM
52 STUDENTS AS S
53 JOIN
54 DEPARTMENTS AS D
55 ON
56 D.DEPT_ID = S.DEPT_ID
57 JOIN
58 MARKS AS M
59 ON
60 M.STUDENT_ID = S.STUDENT_ID
61 GROUP BY D.DEPT_NAME
62 ORDER BY "AVERAGE MARKS" DESC;
63
64

```

department character varying (50)	AVERAGE MARKS numeric
1 Mathematics	95.00
2 Computer Science	83.33

EXPERIMENT -2

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to use JOIN to combine data from multiple tables.
- Learn how to use the AVG() aggregate function to calculate average marks.
- Apply GROUP BY to organize data department-wise.
- Use ORDER BY to sort the results from highest to lowest average marks.
- Generate a meaningful analytical report by integrating student, department, subject, and marks data.

CONCLUSION:

The PostgreSQL query successfully generated a department-wise performance report by joining the relevant tables, calculating the average marks using the **AVG()** function, grouping the data by department, and sorting the results in descending order. This experiment demonstrated the effective use of SQL joins, aggregate functions, grouping, and sorting to perform data analysis and produce useful academic performance reports.

EXPERIMENT -2

Two legacy HR systems (A and B) have separate records of employee salaries. These records may overlap. Management wants to **merge these datasets** and identify **each unique employee** (by EmpID) along with their **lowest recorded salary** across both systems.

Objective

1. Combine two tables A and B.
2. Return each EmpID with their **lowest salary**, and the corresponding **Ename**.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

```
--B
CREATE TABLE A
(
    EmpID INT,
    Ename VARCHAR(50),
    Salary INT
);

-- Create Table B
CREATE TABLE B
(
    EmpID INT,
    Ename VARCHAR(50),
    Salary INT
);

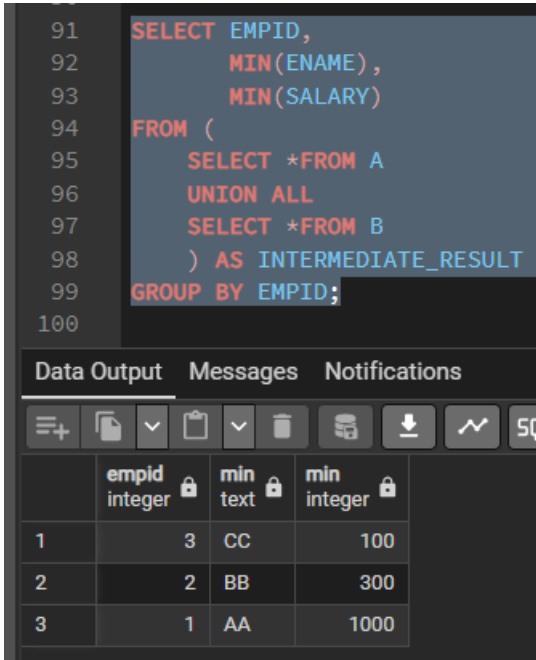
INSERT INTO A (EmpID, Ename, Salary)
VALUES
(1, 'AA', 1000),
(2, 'BB', 300);

INSERT INTO B (EmpID, Ename, Salary)
VALUES
(2, 'BB', 400),
(3, 'CC', 100);
```

EXPERIMENT -2

```
SELECT EMPID,  
       MIN(ENAME),  
       MIN(SALARY)  
FROM (  
    SELECT *FROM A  
    UNION ALL  
    SELECT *FROM B  
    ) AS INTERMEDIATE_RESULT  
GROUP BY EMPID;
```

OUTPUT SCREENSHOTS:



```
91 SELECT EMPID,  
92     MIN(ENAME),  
93     MIN(SALARY)  
94 FROM (  
95     SELECT *FROM A  
96     UNION ALL  
97     SELECT *FROM B  
98     ) AS INTERMEDIATE_RESULT  
99 GROUP BY EMPID;  
100
```

	empid integer	min text	min integer
1	3	CC	100
2	2	BB	300
3	1	AA	1000

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to use JOIN to combine data from multiple tables.
- Learn how to use the AVG() aggregate function to calculate average marks.
- Apply GROUP BY to organize data department-wise.
- Use ORDER BY to sort the results from highest to lowest average marks.

EXPERIMENT -2

- Generate a meaningful analytical report by integrating student, department, subject, and marks data.

CONCLUSION:

The PostgreSQL query successfully generated a department-wise performance report by joining the relevant tables, calculating the average marks using the **AVG()** function, grouping the data by department, and sorting the results in descending order. This experiment demonstrated the effective use of SQL joins, aggregate functions, grouping, and sorting to perform data analysis and produce useful academic performance reports.

EXPERIMENT -2

A multinational organization maintains separate records for employee information and department details. The management wants to identify the employees who receive the highest salary within their respective departments for an annual performance report.

If more than one employee earns the same highest salary in a department, all such employees should be included in the report. The final output should display the **Department Name, Employee Name, and Salary**, arranged in ascending order of department name.

Note: The employee and department information are maintained in separate normalized tables.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

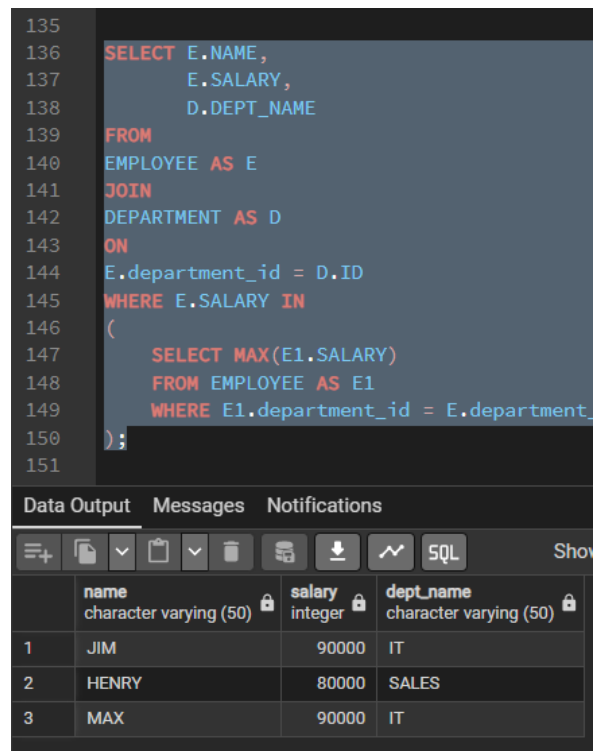
CODE SCREENSHOTS:

```
4
5  --C.
6  CREATE TABLE Department
7  (
8      id INT PRIMARY KEY,
9      dept_name VARCHAR(50)
10 );
11
12 CREATE TABLE Employee
13 (
14     id INT PRIMARY KEY,
15     name VARCHAR(50),
16     salary INT,
17     department_id INT,
18     FOREIGN KEY(department_id)
19     REFERENCES Department(id)
20 );
21
22 INSERT INTO Department
23 VALUES
24 (1, 'IT'),
25 (2, 'SALES');
26
27 INSERT INTO Employee
28 VALUES
29 (1, 'JOE', 70000, 1),
30 (2, 'JIM', 90000, 1),
31 (3, 'HENRY', 80000, 2),
32 (4, 'SAM', 60000, 2),
33 (5, 'MAX', 90000, 1);
34
```

EXPERIMENT -2

```
SELECT E.NAME,  
       E.SALARY,  
       D.DEPT_NAME  
FROM  
EMPLOYEE AS E  
JOIN  
DEPARTMENT AS D  
ON  
E.department_id = D.ID  
WHERE E.SALARY IN  
(  
    SELECT MAX(E1.SALARY)  
    FROM EMPLOYEE AS E1  
    WHERE E1.department_id = E.department_id  
);
```

OUTPUT SCREENSHOTS:



The screenshot shows the SQL Developer interface. The top pane displays the SQL query being executed. The bottom pane shows the 'Data Output' tab with the results of the query. The output table has four columns: 'name', 'salary', and 'dept_name'. The results show three rows of data.

	name	salary	dept_name
1	JIM	90000	IT
2	HENRY	80000	SALES
3	MAX	90000	IT

EXPERIMENT -2

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to retrieve data from multiple normalized tables using JOIN operations.
- Learn to identify employees with the highest salary within each department using SQL queries.
- Apply aggregate functions such as MAX() to determine the maximum salary in each department.
- Use subqueries or equivalent SQL techniques to compare employee salaries with the highest departmental salary.
- Handle situations where multiple employees share the highest salary, ensuring all qualifying employees are included in the result.

CONCLUSION:

The SQL query successfully identified the employee(s) with the highest salary in each department by joining the employee and department tables and using the MAX() aggregate function with a subquery. It correctly included all employees who shared the highest salary within their respective departments and displayed the department name, employee name, and salary in ascending order of department name. This experiment demonstrated the effective use of SQL joins, aggregate functions, subqueries, and sorting techniques to generate meaningful organizational reports.



EXPERIMENT -2